

Guía de Vectorización SIMD

Conversión de Mayúsculas/Minúsculas con 256 bits

Introducción a SIMD

SIMD (Single Instruction, Multiple Data) es una técnica de procesamiento paralelo donde una sola instrucción opera sobre múltiples datos simultáneamente. En lugar de procesar un carácter a la vez como en el código serial, podemos procesar **32 caracteres simultáneamente** usando registros de 256 bits.

Un vector de 256 bits puede contener 32 bytes (caracteres ASCII), permitiendo convertir 32 letras en una sola operación vectorial.

1. Conceptos Clave de Vectorización

Concepto	Descripción	Valor
Vector SIMD	Registro que contiene múltiples datos	256 bits
Caracteres por vector	Cuántos chars caben en 256 bits	32 caracteres
Speedup teórico	Mejora potencial de rendimiento	Hasta 32x
Extensión SIMD	Conjunto de instrucciones usado	AVX2

2. Procesamiento Vectorizado: Paso a Paso

Paso 1: Cargar 32 Caracteres

En lugar de leer un carácter a la vez, cargamos 32 caracteres desde memoria directamente en un registro SIMD de 256 bits:

```
// Cargar 32 bytes desde la posición i del texto __m256i vec =  
_mm256_loadu_si256((__m256i*)&text[i]);
```

Nota: `_mm256_loadu_si256` carga datos **no alineados**. Si los datos están alineados a 32 bytes, se puede usar `_mm256_load_si256` para mejor rendimiento.

Paso 2: Detectar Mayúsculas (Crear Máscara)

Para identificar qué caracteres son mayúsculas, creamos una máscara comparando cada byte del vector con los límites 'A' (65) y 'Z' (90):

```
// Crear valores constantes para comparación __m256i lower_A =  
_mm256_set1_epi8('A' - 1); // 64 __m256i upper_Z = _mm256_set1_epi8('Z' + 1); //  
91 // Crear máscaras de comparación __m256i mask1 = _mm256_cmpgt_epi8(vec,  
lower_A); // vec > 'A'-1 __m256i mask2 = _mm256_cmpgt_epi8(upper_Z, vec); //  
'Z'+1 > vec // Combinar máscaras (AND lógico) __m256i mask =  
_mm256_and_si256(mask1, mask2);
```

La máscara resultante tiene bytes con valor 0xFF (255) donde hay mayúsculas, y 0x00 donde no hay mayúsculas.

Paso 3: Aplicar Conversión Condicional

Sumamos 32 a todos los bytes, pero solo aplicamos el resultado donde la máscara indica que hay una mayúscula:

```
// Crear vector con 32 en cada byte __m256i offset = _mm256_set1_epi8(32); //  
Sumar 32 a todos los caracteres __m256i converted = _mm256_add_epi8(vec, offset);  
// Seleccionar: usar 'converted' donde mask=1, sino usar 'vec' original __m256i  
result = _mm256_blendv_epi8(vec, converted, mask);
```

La instrucción `_mm256_blendv_epi8` es clave: selecciona byte por byte entre dos vectores basándose en la máscara. Esto garantiza que solo las mayúsculas se conviertan.

Paso 4: Almacenar Resultado

Finalmente, escribimos los 32 caracteres procesados de vuelta a memoria:

```
// Almacenar 32 bytes en la posición i _mm256_storeu_si256((__m256i*)&text[i],  
result);
```

3. Pseudocódigo del Algoritmo Completo

El algoritmo vectorizado procesa el texto en bloques de 32 caracteres:

```
case_converter_SIMD_256(text[], m, case_type): // Constantes VECTOR_SIZE = 32 //
Procesar bloques completos de 32 caracteres i = 0 mientras i + VECTOR_SIZE <= m:
// 1. Cargar 32 caracteres vec = CARGAR_VECTOR(text[i]) // 2. Crear máscara de
detección Si case_type == 0: // Convertir a minúsculas lower = 'A' - 1 upper =
'Z' + 1 Sino: // Convertir a mayúsculas lower = 'a' - 1 upper = 'z' + 1 mask1 =
vec > lower mask2 = upper > vec mask = mask1 AND mask2 // 3. Aplicar conversión
offset = Si case_type == 0 entonces 32 sino -32 converted = vec + offset result =
BLEND(vec, converted, mask) // 4. Almacenar ALMACENAR_VECTOR(text[i], result) i =
i + VECTOR_SIZE // Procesar caracteres restantes (menos de 32) mientras i < m: //
Procesamiento serial del resto caracter = text[i] Si es_letra_objetivo(caracter):
text[i] = caracter + offset i = i + 1
```

4. Manejo de Casos Especiales

Caso 1: Texto más corto que 32 caracteres

Si el texto tiene menos de 32 caracteres, hay tres opciones:

- **Opción A:** Procesar todo de forma serial (simple pero menos eficiente)
- **Opción B:** Usar carga con máscara para evitar leer memoria inválida
- **Opción C:** Crear un buffer temporal alineado con padding

Caso 2: Tamaño no múltiplo de 32

Cuando el texto no es múltiplo exacto de 32, quedan caracteres residuales. Por ejemplo, un texto de 50 caracteres se procesaría así:

- Caracteres 0-31: Procesamiento SIMD (32 chars)
- Caracteres 32-49: Procesamiento serial o con máscara (18 chars)

```
// Después del bucle vectorizado while (i < m) { char c = text[i]; if (case_type == 0 && c >= 'A' && c <= 'Z') { text[i] = c + 32; } else if (case_type == 1 && c >= 'a' && c <= 'z') { text[i] = c - 32; } i++; }
```

Caso 3: Datos No Alineados

En la práctica, los datos raramente están alineados a 32 bytes. Por eso usamos las instrucciones 'unaligned' (*loadu/storeu*) que funcionan con cualquier dirección de memoria, aunque con una pequeña penalización de rendimiento.

5. Comparación: Serial vs SIMD

Aspecto	Serial	SIMD (256 bits)
Chars por iteración	1	32
Iteraciones para 1000 chars	1000	~32
Instrucciones por char	~5-7	~0.3
Speedup teórico	1x	32x
Complejidad código	Simple	Media
Portabilidad	100%	Requiere AVX2

6. Lista de Intrinsics Necesarios

Para implementar el algoritmo SIMD, necesitas los siguientes intrinsics de AVX2:

Intrinsic	Descripción	Uso
_mm256_loadu_si256	Carga 256 bits no alineados	Leer 32 chars
_mm256_storeu_si256	Almacena 256 bits no alineados	Escribir 32 chars
_mm256_set1_epi8	Crea vector con valor repetido	Constantes (32, A, Z)
_mm256_cmpgt_epi8	Compara > byte a byte	Detectar rangos
_mm256_and_si256	AND lógico de vectores	Combinar máscaras
_mm256_add_epi8	Suma byte a byte	Aplicar offset
_mm256_sub_epi8	Resta byte a byte	Mayúsculas
_mm256_blendv_epi8	Selección condicional	Aplicar conversión

Headers necesarios:

```
#include <immintrin.h> // Para intrinsics AVX2 #include <vector> #include <cstdint>
```

7. Consideraciones de Rendimiento

Cuellos de Botella Potenciales:

- **Ancho de banda de memoria:** Si la memoria no puede proveer datos lo suficientemente rápido, las unidades SIMD esperan
- **Latencia de caché:** Cache misses causan grandes penalizaciones de rendimiento
- **Alineamiento:** Datos no alineados tienen penalizaciones (~10-20% más lentos)
- **Branch prediction:** En el procesamiento del resto, las ramas pueden ser impredecibles

Optimizaciones Recomendadas:

- Usar datos alineados a 32 bytes cuando sea posible
- Procesar datos en bloques grandes para aprovechar caché
- Minimizar saltos condicionales en el bucle principal
- Considerar prefetching para grandes volúmenes de datos

8. Escalabilidad a 512 bits (AVX-512)

Si el hardware soportara vectores de 512 bits, podríamos procesar **64 caracteres simultáneamente**, duplicando el speedup teórico a 64x. Esto requeriría:

- Cambiar de `_mm256_` a `_mm512_` intrinsics
- Ajustar `VECTOR_SIZE` de 32 a 64
- Usar `__m512i` en lugar de `__m256i`
- Verificar soporte del CPU con `CPUID`

Nota: AVX-512 no está disponible en todos los procesadores y puede causar throttling térmico en algunos CPUs.

9. Conclusión y Próximos Pasos

La vectorización SIMD transforma el procesamiento serial carácter por carácter en procesamiento paralelo de 32 caracteres simultáneos, ofreciendo mejoras significativas de rendimiento.

Ventajas principales:

- Speedup de hasta 32x en casos ideales
- Reducción dramática del número de iteraciones
- Mejor utilización de los recursos del procesador
- Escalabilidad a vectores más grandes (AVX-512)

Desafíos a considerar:

- Complejidad adicional en el código
- Manejo cuidadoso de casos especiales
- Dependencia del hardware (portabilidad)
- Debugging más difícil

El siguiente paso es implementar este algoritmo en C++ usando los intrinsics descritos y realizar mediciones de rendimiento comparativas con la versión serial.